

Project Plan

Weather Data Pipeline with Python and PostgreSQL

GitHub: <https://github.com/mohamed-mahmoud-de/Weather-Data-Pipeline>

1. Project Description

This project aims to design and implement an end-to-end data engineering pipeline that extracts weather data from a public API, processes and cleans the data using Python, and stores it in a relational PostgreSQL database for analytical use.

The project focuses on applying core Data Engineering concepts such as API ingestion, data transformation, schema design, batch processing, containerization, automation, and monitoring using industry-standard tools and best practices.

2. Technologies and Tools

- **Programming Language:** Python
- **Data Source:** Open-Meteo Weather API
- **Data Format:** JSON
- **Database:** PostgreSQL
- **Containerization:** Docker & Docker Compose
- **Scheduling / Automation:** Cron or Apache Airflow
- **Version Control:** Git & GitHub

3. Project Objectives

- Build a reliable and maintainable ETL pipeline.
- Ingest real-world weather data from a public API.
- Clean, normalize, and transform JSON data.
- Design and implement a relational PostgreSQL schema.
- Store structured data efficiently for analysis.
- Automate pipeline execution and monitor reliability.
- Produce clear documentation and a working demo.

4. Project Milestones and Tasks

Milestone 1: Data Collection, Exploration, and Preprocessing

Objective: Collect raw weather data and understand its structure and quality.

Tasks:

- Select a public weather API (e.g., Open-Meteo).
- Fetch raw weather data in JSON format.
- Explore API response structure and available fields.
- Identify data quality issues such as missing values.
- Document preprocessing requirements.

Deliverables: Raw JSON data samples, data exploration summary.

Milestone 2: System Development and Data Transformation

Objective: Clean, transform, and prepare weather data for database storage.

Tasks:

- Implement ETL scripts in Python.
- Normalize nested JSON data into tabular format.
- Handle missing or invalid values.
- Apply transformations (timestamps, units, locations).
- Design PostgreSQL tables and schema.
- Validate data correctness and consistency.

Deliverables: Python ETL scripts, PostgreSQL schema design, transformed datasets.

Milestone 3: Deployment (Batch Processing)

Objective: Deploy the ETL pipeline as a batch process.

Tasks:

- Set up PostgreSQL using Docker.
- Containerize the Python ETL application.
- Load transformed data into PostgreSQL tables.
- Schedule pipeline execution using Cron or Apache Airflow.
- Verify successful data insertion.

Deliverables: Dockerized ETL pipeline, populated PostgreSQL database.

Milestone 4: Automation, Monitoring, and Reliability

Objective: Ensure pipeline automation, stability, and observability.

Tasks:

- Automate scheduled pipeline runs.
- Implement logging for each pipeline step.
- Monitor execution status and failures.
- Handle retries and error recovery.
- Track data freshness.

Deliverables: Automated ETL workflow, logs, monitoring setup.

Milestone 5: Final Documentation, Demo, and Presentation

Objective: Deliver a complete and well-documented project.

Tasks:

- Document ETL architecture and workflow.
- Describe PostgreSQL schema and tables.
- Prepare a live demo of pipeline execution.
- Run analytical SQL queries.
- Prepare final project presentation.

Deliverables: Project documentation, GitHub repository, demo and presentation.

5. Final Project Deliverables

- End-to-end weather data ETL pipeline.
- Python source code and configuration files.
- Docker and Docker Compose setup.
- PostgreSQL database with structured weather data.
- Project documentation.
- GitHub repository.
- Final presentation.

6. Project Value

This project demonstrates practical Data Engineering skills including API-based data ingestion, ETL development, relational data modeling, containerization, automation, and monitoring. It reflects real-world batch data pipeline design and prepares the student for entry-level Data Engineering roles.